

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-  
a2c643d6f89e7ffde.jsonl`  
- SHA-256: `244123870c91f4c297bb659b8179467331f7745f398bcca99b6d04c514f48dbf`  
- Source modified: `2026-07-06T09:16:06+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_3512746c-54e`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:12:51.501Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +
an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY:
command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate),
tests/test_command_service.py (10 tests).
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel
creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND docs/adr/0009-selective-channel-scanning.md, trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff
 and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur /
 pre-existing. PLAUSIBLE if unsettled. Corrected severity.
 FINDING (commands): /watch and /unwatch by @username or channel name are non-functional for real Telegram sources
 severity high | src/tg_video_filter/command_service.py:410
 summary: _resolve_source only looks up get_source_by_provider_id, but real Telegram sources have a NUMERIC provider_source_id (str(channel_id)), never a username or title ? so ADR 0009's '/watch <#name|id>' (line 64) resolves only by numeric id; @username and display-name both return 'not found' for a known channel.
 scenario: A real source exists with provider_source_id='-1001234567890', title='Classical Music Daily', watched=False. User runs '/watch @classicalmusic' -> '? Source `@classicalmusic` not found.' User runs '/watch Classical Music Daily' -> '? ... not found.' Only '/watch -1001234567890' succeeds. Since a normal user knows a channel by its @handle or its name and rarely by its numeric internal id, the ergonomic path the ADR specifies is broken; users cannot watch/unwatch a channel by the identifiers they actually have.
 evidence: _resolve_source (command_service.py:404-420) does target.lstrip('@') then db.get_source_by_provider_id(conn, user_id, 'telegram', target), with a fallback of '@'+target ? both are provider_source_id lookups. There is NO query against the title column or a username/handle column. upsert_channel (db.py:1399) and upsert_source always set provider_source_id=str(channel_id) (numeric). Reproduced with a realistic seed: '/watch @classicalmusic' and '/watch Classical Music Daily' both return 'not found'; only '/watch -1001234567890' works. The passing test test_watch_by_username (tests/test_command_service.py:301-303) only passes because it seeds an UNREALISTIC source with provider_source_id='@mychan', which never occurs in production (upsert_channel/on_new_message always write str(channel_id)). ADR 0009 line 64 requires '/watch <#name|id>' (name OR id).
 rec: Resolve by name/username in addition to numeric id: add a title/username lookup in _resolve_source (e.g. a case-insensitive match on sources.title and/or a dedicated username column) so '/watch <name>' and '/watch @handle' work against real numeric-id sources. Re-seed test_watch_by_username with a numeric provider_source_id plus a title/username to reflect production, so the test actually exercises name resolution rather than a fictitious id.

2. assistant (2026-07-06T09:12:54.973Z)

I'll verify this finding by reading the actual code and the ADR.

3. user (2026-07-06T09:12:56.618Z)

```

380         lines = ["? **All sources**", ""]
381         for s in sources:
382             icon = "?" if s.watched else ""
383             label = s.title or s.provider_source_id
384             lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
385         return CommandResult("\n".join(lines))
386
387     async def _handle_watchlist(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
388         """List only watched sources."""
389         sources = await db.list_account_sources(conn, user_id, watched_only=True)
390         if not sources:
391             return CommandResult(
392                 "? **No watched sources.**\n\n"
393                 "Use `/channels` to see available sources, then `/watch <id>` "
394                 "to start scanning."
395             )
396         lines = ["? **Watched sources**", ""]
397         for s in sources:
398             label = s.title or s.provider_source_id
399             lines.append(f"? `{s.provider_source_id}` ? {label}")
400         return CommandResult("\n".join(lines))
401
402     async def _resolve_source(
403         self, conn: aiomysqlite.Connection, user_id: int, target: str

```

```

404 ) -> Source | None:
405     """Resolve a user-provided source identifier to a ``Source`` row.
406
407     Accepts ``@username`` or a numeric channel id string. The ``@`` prefix
408     is stripped before lookup since Telegram channel ids are numeric.
409     """
410     target = target.lstrip("@")
411     # Try by provider-native id first (the common case).
412     src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
413     if src is not None:
414         return src
415     # Fall back: try with @ prefix in the target column (just in case).
416     return await db.get_source_by_provider_id(conn, user_id, "telegram",
f"@{target}")
417
418     # -- internal -----
419
420     async def _update(
421         self,
422         conn: aiosqlite.Connection,
423         user_id: int,
424         current: FilterConfig,
425         max_size_mb: int | None = None,
426         max_duration_s: int | None = None,
427         min_duration_s: int | None = None,
428         target_height: int | None = None,
429         exclude_gif: bool | None = None,
430         exclude_round: bool | None = None,
431         exclude_nosound: bool | None = None,
432     ) -> FilterConfig:
433         """Apply partial updates to the account's config, persist, and return."""
434         updates: dict[str, object] = {}
435         if max_size_mb is not None:
436             updates["max_size_mb"] = max_size_mb
437         if max_duration_s is not None:
438             updates["max_duration_s"] = max_duration_s
439         if min_duration_s is not None:
440             updates["min_duration_s"] = min_duration_s
441         if target_height is not None:
442             updates["target_height"] = target_height
443         if exclude_gif is not None:
444             updates["exclude_gif"] = exclude_gif
445         if exclude_round is not None:
446             updates["exclude_round"] = exclude_round
447         if exclude_nosound is not None:
448             updates["exclude_nosound"] = exclude_nosound
449         cfg = current.model_copy(update=updates) if updates else current
450         await db.save_filter_config(conn, user_id, cfg)
451         log.info(
452             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
453             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
454             user_id,
455             cfg.max_size_mb,
456             cfg.min_duration_s,
457             cfg.max_duration_s,
458             cfg.target_height,
459             cfg.exclude_gif,

```

4. user (2026-07-06T09:12:56.860Z)

```

1     # ADR 0009: Selective channel scanning and multiplexed routing
2
3     - **Status**: Accepted
4     - **Date**: 2026-07-04
5     - **Decision owner**: project owner

```

```

6
7  ## Context
8
9  Today the userbot scans every channel/group the account is a member of.
10 `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11 filter, and `on_new_message` processes every video regardless of source. The
12 `channels` table exists but is populated lazily as a record of seen channels;
13 it is not used as a filter.
14
15 The product vision is a multiplexing video re-distribution system:
16
17 - A small set of high-quality source channels (e.g. spiritual, classical
18   music) are scanned for incoming videos.
19 - Matches are routed to themed destination broadcast channels (e.g. a
20   Reggae channel, a Rap channel) that subscribers opt into.
21 - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23 This requires two capabilities the current architecture does not have:
24
25 1. Selective scanning ? choose which source channels are watched, rather
26   than ingesting all of them.
27 2. Per-source ? per-destination routing ? a match from source channel X is
28   delivered to a specific destination channel Y, instead of the single global
29   `delivery_target` (ADR 0005).
30
31  ## Decision
32
33  Add three additive, independently-shippable capabilities:
34
35  ### 1. Selective scanning (watched-flag)
36
37  Add a `watched` flag to the `channels` table. The default is locked as
38  opt-in: `DEFAULT 0`, so a freshly-onboarded user scans nothing until they
39  explicitly `/watch` a channel. This matches the multiplexing product shape
40  (sources are a curated set) and avoids blindly re-ingesting every chat the
41  account is a member of.
42
43  Gate `on_new_message` at the top: after computing `channel_id`, look up the
44  row and `return` early if `watched` is false, before any dedup or filter work.
45
46  This is a small migration plus ~5 lines in `telethon_client.py`.
47
48  Discoverability note. With `DEFAULT 0` plus the current lazy population
49  (a `channels` row is only inserted when a message is first seen), opt-in is
50  necessary but not sufficient for discoverability: a channel the account has
51  never received a message from has no row to flag. The `/channels` command
52  (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53  so users can browse and watch channels they have not yet received traffic
54  from. The flag default and the enumeration flow are orthogonal ? both are
55  required.
56
57  ### 2. Channel enumeration and management commands
58
59  New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60  existing `commands.py` pattern:
61
62  - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63    used in `backfill.py`) with their watched state.
64  - `/watch <#name|id>` ? set `watched=1` for a source channel.
65  - `/unwatch <#name|id>` ? set `watched=0`.
66  - `/watchlist` ? list only watched sources.
67
68  ### 3. Per-source ? per-destination routing rules
69
70  Introduce a `routing_rules` table (the canonical name; issue #25's `routes`

```

```

71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ...
74 routing_rules (
75     user_id          INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id INTEGER NOT NULL,
77     dest_channel_id  INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id INTEGER,             -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )
81 ...
82
83 On a match, look up applicable routing rules by `source_channel_id` and
84 deliver to each matching destination. The delivery layer (`delivery.py`)
85 already accepts a `target` per call, so this generalises the existing single
86 `delivery_target` to a per-match lookup. Users with no routing rules fall
87 back to the current global `delivery_target` behaviour (ADR 0005), preserving
88 backwards compatibility.
89
90 ## Consequences
91
92 **Positive**
93 - Enables the multiplexing product shape: scan curated sources, publish to
94   topical broadcast channels, let subscribers pick outputs by interest.
95 - Each capability is additive and independently deployable; none disturbs the
96   existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97 - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original*
98   file, so a routed re-upload is not re-processed.
99 - Channel enumeration reuses `client.get_dialogs()` already present in
100  `backfill.py`; no new Telethon capability is required.
101
102 **Negative**
103 - New schema (`channels.watched`, `routing_rules`) with migrations.
104 - **Routing loop invariant.** With multiplexed routing, the userbot account
105   may be a member of a *destination* broadcast channel and re-see its own
106   output, producing an infinite forward loop. The contract is: **a channel
107   is either a source or a destination, never both.** This is enforced in
108   `on_new_message` by excluding any channel that is a configured destination
109   (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110   user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111   this; the invariant is stated here so the handler has a contract to code
112   against.
113 - Per-route filter overrides add config complexity; v1 may ship with global
114   filters only and a single implicit route (`source ? global target`), then
115   layer explicit per-route rules later.
116
117 ## Backwards compatibility
118
119 Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120 install from receiving matches until the user re-watches their channels ? a
121 behaviour change disguised as a schema change. To preserve the current
122 "ingest everything" semantics for existing installs, the migration backfills:
123
124 ```sql
125 ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126 UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127 ```
128
129 i.e. any channel the install has *already* seen traffic from is implicitly
130 watched, so pre-migration users observe no change. New channels discovered
131 after the migration default to `watched = 0` and must be explicitly watched ?
132 which is the intended new UX. This mirrors the "old rows behave exactly as
133 before" pattern established by ADR 0005.
134
135 ## Rollout order

```

```

136
137 1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138    `/channels`). Smallest unit; immediately useful even without routing.
139 2. **Named destination channels** ? extend `/set delivery_target` to allow
140    switching among a user-defined set, not just one global value.
141 3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142    `/route <source> <dest>` command). The full multiplexing vision.
143
144 Each step is shippable and useful on its own; step 3 is not required to
145 deliver value from steps 1?2.
146
147 ## Alternatives considered
148
149 - **Server-side `chats=` filter at handler registration:** rejected ? the
150   watched set is per-user and mutable at runtime; re-registering Telethon
151   handlers on every `/watch` change is brittle. An in-handler lookup against
152   the `channels` table is simpler and hot-swappable.
153 - **Content-based classification (auto-detect genre):** explicitly out of
154   scope for v1. Routing is user-defined by source channel; automatic topic
155   detection could be a future enhancement but is not required to deliver the
156   multiplexing UX.
157
158 ## References
159
160 - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161 - ADR 0005 (per-user delivery target) ? the single-target delivery this
162   generalises into per-route targets.
163 - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164   the original `document_id`.
165 - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166   `/unwatch`, and routing commands to public users.
167

```

5. assistant (2026-07-06T09:12:59.776Z)

Now I understand the source model. Let me verify the key claims: how sources get their `provider\_source\_id`, and reproduce the resolution behavior.

6. user (2026-07-06T09:13:01.430Z)

```

94- account_id          INTEGER NOT NULL REFERENCES accounts(id),
95- provider            TEXT NOT NULL DEFAULT 'telegram',
96- provider_source_id  TEXT NOT NULL,
97- title               TEXT,
98- kind                TEXT NOT NULL DEFAULT 'channel',
99- --
100- seen_first_at       TEXT NOT NULL,
101- last_seen_at        TEXT,
102- UNIQUE (account_id, provider, provider_source_id)
103-);
104-
105- --
106- 526-      ""
107- 527-      INSERT OR IGNORE INTO sources
108- 528-          (account_id, provider, provider_source_id, title, kind, watched,
109- 529-            seen_first_at, last_seen_at)
110- 530-      SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
111- 531- --
112- 541-      ""
113- 542-      INSERT OR IGNORE INTO sources
114- 543-          (account_id, provider, provider_source_id, kind, watched, seen_first_at)
115- 544-      SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
116- 545-          'channel', 0, MIN(v.seen_at)
117- 546- --
118- 576-      ON s.account_id = v.user_id

```

```

577-         AND s.provider = 'telegram'
578-         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
579-         ""
580-     )
581- --
750-     JOIN sources sc
751-         ON sc.id = ci.source_id
752-         AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
753-     JOIN destinations d ON d.account_id = vd.user_id
754-     JOIN routing_rules rr
755- --
778-
779-def _safe_int_cast(value: object, field_name: str) -> int:
780-     """Cast a provider_source_id / provider_message_id / document_id to int.
781-
782-     All existing data is Telegram ? these ids are numeric strings. Non-Telegram
783-     --
846-         account_id=row["account_id"],
847-         provider=row["provider"],
848-         provider_source_id=row["provider_source_id"],
849-         title=row["title"],
850-         kind=row["kind"],
851-         --
1105-     """Insert a source row, or update title/watched on conflict.
1106-
1107-     The conflict key is ``(account_id, provider, provider_source_id)`` per the
1108-     ADR 0011 schema. Returns the persisted row.
1109-     """
1110-     await conn.execute(
1111-         """INSERT INTO sources
1112-             (account_id, provider, provider_source_id, title, kind, watched,
1113-              seen_first_at, last_seen_at)
1114-             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1115-             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1116-                 title=excluded.title,
1117-                 watched=excluded.watched,
1118-             --
1119-                 source.account_id,
1120-                 source.provider,
1121-                 source.provider_source_id,
1122-                 source.title,
1123-                 source.kind,
1124-             --
1125-         """
1126-     )
1127-
1128-     async with conn.execute(
1129-         """SELECT * FROM sources
1130-             WHERE account_id = ? AND provider = ? AND provider_source_id = """,
1131-         (source.account_id, source.provider, source.provider_source_id),
1132-     ) as cur:
1133-         row = await cur.fetchone()
1134-         --
1141-
1142-async def get_source_by_provider_id(
1143-    conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144-) -> Source | None:
1145-     """Resolve a source by its provider-native id."""
1146-     --
1147-     async with conn.execute(
1148-         """SELECT * FROM sources
1149-             WHERE account_id = ? AND provider = ? AND provider_source_id = """,
1150-         (account_id, provider, provider_source_id),
1151-     ) as cur:
1152-         row = await cur.fetchone()
1153-         --
1397-         account_id=channel.user_id,
1398-         provider="telegram",

```

```

1399:         provider_source_id=str(channel.channel_id),
1400:         title=channel.title,
1401:         kind="channel",
1402:         --
1429:         id=src.id,
1430:         user_id=src.account_id,
1431:         channel_id=int(src.provider_source_id),
1432:         title=src.title,
1433:         seen_first_at=src.seen_first_at,
1434:         --
1465:     if src is None:
1466:         return None
1467:     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1468:
1469:     --
1492:     conn.row_factory = aiomysqlite.Row
1493:     async with conn.execute(
1494:         """SELECT ci.*, s.provider_source_id AS src_pid
1495:            FROM content_items ci
1496:           JOIN sources s ON s.id = ci.source_id
1497:           --
1509:           id=row["id"],
1510:           user_id=user_id,
1511:           channel_id=_safe_int_cast(row["src_pid"], "provider_source_id"),
1512:           message_id=_safe_int_cast(row["provider_message_id"], "provider_message_id"),
1513:           document_id=(
1514:           --
1555:     if watched_only:
1556:         query += " AND watched = 1"
1557:     query += " ORDER BY watched DESC, provider_source_id ASC"
1558:     async with conn.execute(query, params) as cur:
1559:         rows = await cur.fetchall()
1560:         --
1574:         account_id=video.user_id,
1575:         provider="telegram",
1576:         provider_source_id=str(video.channel_id),
1577:         kind="channel",
1578:         watched=True,
1579:         --
1697:     conn.row_factory = aiomysqlite.Row
1698:     async with conn.execute(
1699:         """SELECT ci.*, s.provider_source_id AS src_pid
1700:            FROM content_items ci
1701:           JOIN sources s ON s.id = ci.source_id
1702:           --
1709:           id=r["id"],
1710:           user_id=user_id,
1711:           channel_id=_safe_int_cast(r["src_pid"], "provider_source_id"),
1712:           message_id=_safe_int_cast(r["provider_message_id"], "provider_message_id"),
1713:           document_id=(

```

7. user (2026-07-06T09:13:01.916Z)

```

290
291
292     async def test_watch_by_numeric_id(conn: aiomysqlite.Connection, svc: CommandService) ->
293     None:
294         await _seed_source(conn, USER_ID, "-1001", "Target", watched=False)
295         result = await svc.handle(USER_ID, "/watch -1001", conn, FilterConfig())
296         assert "Now watching" in result.reply_text
297         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
298         assert src is not None
299         assert src.watched is True

```

```

300
301     async def test_watch_by_username(conn: aisqlite.Connection, svc: CommandService) ->
None:
302         await _seed_source(conn, USER_ID, "@mychan", "My Channel", watched=False)
303         result = await svc.handle(USER_ID, "/watch @mychan", conn, FilterConfig())
304         assert "Now watching" in result.reply_text
305         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "@mychan")
306         assert src is not None
307         assert src.watched is True
308
309
310     async def test_unwatch(conn: aisqlite.Connection, svc: CommandService) -> None:
311         await _seed_source(conn, USER_ID, "-1001", "Target", watched=True)
312         result = await svc.handle(USER_ID, "/unwatch -1001", conn, FilterConfig())
313         assert "Stopped watching" in result.reply_text
314         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
315         assert src is not None
316         assert src.watched is False
317
318
319     async def test_watch_unknown_source_shows_error(
320         conn: aisqlite.Connection, svc: CommandService
321     ) -> None:
322         result = await svc.handle(USER_ID, "/watch -9999", conn, FilterConfig())
323         assert "not found" in result.reply_text
324
325
326     async def test_set_source_watched_toggle(conn: aisqlite.Connection) -> None:
327         src = await _seed_source(conn, USER_ID, "-2000", watched=True)
328         assert src.id is not None
329         await db.set_source_watched(conn, USER_ID, src.id, False)

```

8. assistant (2026-07-06T09:13:07.501Z)

The claim is confirmed on all structural facts: `upsert\_channel` (db.py:1399) and the video upsert (db.py:1576) always write `provider\_source\_id=str(channel\_id)` ? numeric. `\_resolve\_source` only queries `get\_source\_by\_provider\_id` which matches on `provider\_source\_id` column, never `title`. The `@`-fallback just re-queries the same column.

Let me run a repro to confirm the real-world behavior with a numeric-id source.

9. user (2026-07-06T09:15:13.533Z)

Exit code 143

Command timed out after 2m 0s

Traceback (most recent call last):

```

File "C:\Users\avidu\AppData\Local\Temp\repro_watch.py", line 24, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
File "C:\Users\avidu\AppData\Local\Temp\repro_watch.py", line 11, in main
    await db.upsert_account(conn, provider="telegram", provider_account_id="42", title="me")
    ~~~~~^~~~~~
TypeError: upsert_account() got an unexpected keyword argument 'provider'

```

10. assistant (2026-07-06T09:15:16.531Z)

Let me check the actual signatures for upsert_account and how _seed_source works in the tests.

11. user (2026-07-06T09:15:17.125Z)

```
1      """Tests for the provider-independent CommandService (command_service.py).
2
3      These tests exercise command parsing and config mutation without any
4      Telethon objects ? pure strings in, CommandResult out. The Telethon
5      adapter (commands.py) is tested separately via test_commands.py.
6      """
7
8      from __future__ import annotations
9
10     import aiosqlite
11     import pytest
12
13     from tg_video_filter import db
14     from tg_video_filter.command_service import (
15         CommandService,
16         format_commands,
17         parse_duration_s,
18         parse_size_mb,
19     )
20     from tg_video_filter.models import FilterConfig, Source
21
22     USER_ID = 111
23
24
25     @pytest.fixture
26     async def conn() -> aiosqlite.Connection:
27         """An in-memory SQLite connection with a seeded user and default config."""
28         c = await aiosqlite.connect(":memory:")
29         await db._apply_schema(c)
30         await db.upsert_user(
31             c,
32             db.User(id=USER_ID, phone="+15555550100"), # type: ignore[attr-defined]
33         )
34         await db.save_filter_config(c, USER_ID, FilterConfig())
35         yield c
36         await c.close()
37
38
39     @pytest.fixture
40     def svc() -> CommandService:
41         return CommandService()
42
43
44     # --- unit parser tests -----
45
46
47     @pytest.mark.parametrize(
48         "value, unit, expected",
49         [
50             ("10", "MB", 10),
51             ("1", "GB", 1024),
52             ("500", "KB", 1),
53             ("5", "", 5),
54             ("2", "TB", 2 * 1024 * 1024),
55         ],
56     )
57     def test_parse_size_mb(value: str, unit: str, expected: int) -> None:
58         assert parse_size_mb(value, unit) == expected
59
```

```

60
61     def test_parse_size_mb_unknown_unit_raises() -> None:
62         with pytest.raises(ValueError):
63             parse_size_mb("10", "XB")
64
65
66     @pytest.mark.parametrize(
67         "value, unit, expected",
68         [
69             ("15", "m", 900),
70             ("2", "h", 7200),
71             ("90", "s", 90),
72             ("5", "", 5),
73         ],
74     )
75     def test_parse_duration_s(value: str, unit: str, expected: int) -> None:
76         assert parse_duration_s(value, unit) == expected
77
78
79     def test_parse_duration_s_unknown_unit_raises() -> None:
80         with pytest.raises(ValueError):

```

12. assistant (2026-07-06T09:15:21.044Z)

Let me see `\_seed\_source` helper to mirror the test's setup exactly.

13. user (2026-07-06T09:15:22.886Z)

```

242:async def _seed_source(
243-    conn: aiosqlite.Connection,
244-    account_id: int,
245-    provider_source_id: str,
246-    title: str = "",
247-    watched: bool = False,
248-) -> Source:
249-    return await db.upsert_source(
250-        conn,
251-        Source(
252-            account_id=account_id,
253-            provider="telegram",
254-            provider_source_id=provider_source_id,
255-            title=title,
256-            kind="channel",
257-            watched=watched,
258-        ),
259-    )
260-
261-
265:    await _seed_source(conn, USER_ID, "-1001", "Channel A", watched=True)
266:    await _seed_source(conn, USER_ID, "-1002", "Channel B", watched=False)
267:    result = await svc.handle(USER_ID, "/channels", conn, FilterConfig())
268:    assert "Channel A" in result.reply_text
269:    assert "Channel B" in result.reply_text
270:    assert "?" in result.reply_text # watched indicator
271:    assert "?" in result.reply_text # unwatched indicator
272-
273-
274:async def test_channels_empty_shows_help(conn: aiosqlite.Connection, svc: CommandService) ->
None:

```

[Showing results with pagination = limit: 30]

14. assistant (2026-07-06T09:15:26.520Z)

Now I can write a faithful repro using the same fixtures approach.

15. user (2026-07-06T09:15:32.886Z)

```
'/watch @classicalmusic' -> ? Source `@classicalmusic` not found. Use `/channels` to list
available sources.
'/watch Classical Music Daily' -> ? Source `Classical Music Daily` not found. Use `/channels` to
list available sources.
'/watch -1001234567890' -> ? Now watching **Classical Music Daily** (`-1001234567890`).
```

16. assistant (2026-07-06T09:15:41.312Z)

The repro confirms the finding exactly. With a realistic numeric-id source, `/watch @classicalmusic` and `/watch Classical Music Daily` both return "not found"; only the numeric id works.

Now the critical question for CONFIRMED vs REFUTED: does this **deviate** from the accepted ADR 0009? ADR line 64 says `/watch <#name|id>`. Let me weigh whether `<<#name|id>` genuinely mandates name/username resolution or is loose notation.

The ADR consistently frames the syntax as `/watch <#name|id>` (line 64) and `/unwatch <#name|id>` (line 65). The `<<#name|id>` notation explicitly offers two alternatives: a `#name` (channel name/handle) OR an `id`. The code implements only the `id` branch. The `@`-fallback is cosmetic ? it queries the same `provider\_source\_id` column with an `@` prefix, which never matches numeric ids. So name/handle resolution is genuinely absent, and this is a real deviation from the accepted spec.

Let me confirm there's no username column in the sources schema that could make handle-resolution possible, and check the docstring's own claim.

17. user (2026-07-06T09:15:42.437Z)

```
90      -- the delivery logic that populates them.
91
92      CREATE TABLE IF NOT EXISTS sources (
93          id                INTEGER PRIMARY KEY AUTOINCREMENT,
94          account_id        INTEGER NOT NULL REFERENCES accounts(id),
95          provider          TEXT NOT NULL DEFAULT 'telegram',
96          provider_source_id TEXT NOT NULL,
97          title             TEXT,
98          kind              TEXT NOT NULL DEFAULT 'channel',
99          watched           INTEGER NOT NULL DEFAULT 0,
100         seen_first_at     TEXT NOT NULL,
101         last_seen_at      TEXT,
102         UNIQUE (account_id, provider, provider_source_id)
103     );
104
```

18. assistant (2026-07-06T09:15:54.754Z)

Confirmed: the `sources` table has a `title` column but no username/handle column, and `\_resolve\_source` never queries `title`. Real sources always carry a numeric `provider\_source\_id`. So both the `#name`/title path and the `@handle` path of ADR line 64's `/watch <#name|id>` are non-functional; only numeric id works.

Adversarial refutation attempts, all failed:

- "Maybe some code path writes a `@username` into provider_source_id" ? No. Every writer (`upsert\_channel` db.py:1399, video upsert db.py:1576, the `INSERT OR IGNORE ... CAST(channel\_id AS TEXT)` at 530/544) writes `str(channel\_id)`, numeric.
- "Maybe the `@`-prefix fallback resolves handles" ? No. It queries the same `provider\_source\_id` column with `@`+target, which never matches a numeric id.
- "Maybe `<<#name|id>` is loose notation for id-only" ? No. The `|` explicitly separates two alternatives; `#name` denotes a name. The docstring even claims it "Accepts `@username`," which it does not functionally do for real data.

- "Pre-existing?" ? `\_resolve\_source` and these commands are new in this PR's diff.

Severity assessment: The numeric-id path works, so `/watch`/`/unwatch` are usable, but only via the identifier a normal user is least likely to know. The ergonomic path the ADR specifies (name/handle) is broken ? a real ADR-contract violation users hit routinely. High is the correct severity (not blocker: the feature is not fully broken since numeric id works; not medium: this is the primary intended UX, not an edge case).

19. user (2026-07-06T09:16:06.634Z)

Structured output provided successfully